

Java 27, Java'ya Yeni Başlayan Programcılar İçin

Başka bir dilde kod yazan ve şimdi Java'ya ihtiyacı olan geliştiriciler için kendi hızında ilerleyen bir müfredat. Değişkenin ya da döngünün ne olduğunu yeniden anlatmaz. Java'nın bu işleri nasıl yaptığını ve sizi nerede şaşırtacağını anlatır.

Yirmi iki modül. Her birinde bir ders, yorumları *nedenini* açıklayan çalıştırılabilir örnekler, özet çıkarımlar ve referans çözümüyle birlikte bir ödev var.

Bu depodaki her örnek JDK 27 üzerinde çalıştırıldı. Derslerde alıntılanan her derleyici hatası, bozuk kodu gerçekten derleyerek üretildi ve her sayı hatırlanarak değil ölçülerek yazıldı.

Bu, müfredatın Türkçe sürümüdür. İngilizce özgün metinler bir üst dizindedir. Neyin çevrilip neyin çevrilmediği [CEVIRI-NOTLARI.md](#) dosyasında açıklanmıştır. Kısaca: kod, komutlar ve derleyici çıktıları İngilizce bırakıldı.

Burada "Java 27" ne demek

[JDK 27](#) 15 Eylül 2026'da genel kullanıma açıldı. LTS **değil**, ve kesinleşen özellikleri neredeyse tamamen çalışma zamanı seviyesinde: varsayılan çöp toplayıcı olarak G1, küçültülmüş nesne başlıkları, post-kuantum TLS anahtar değişimi, uçuş kaydedici maskeleme. Bu listede Java yazma biçiminizi değiştiren hiçbir şey yok.

Bu yüzden müfredat kendi başlığı konusunda dürüst:

JDK 27 çalışma ortamı ve araç zinciridir. Öğrendiğiniz dil ise [JDK 25 LTS](#) sürümüne kadar birikmiş kararlı Java'dır.

Modül 22, JDK 26 ve 27'nin gerçekte neyi değiştirdiğini, LTS olmayan sürümlerin neden sürüm numarasının düşündüğünden daha az önemli olduğunu ve bir JEP'i kendi başınıza nasıl okuyacağınızı anlatır.

Başlarken

Tek gereksinim JDK 27. Maven yok, Gradle yok, bu depoda hiçbir üçüncü parti kütüphane yok.

```
java -version # görmeniz gereken: java version "27"
```

Kurulum

Platform	Komut
macOS (Homebrew)	<code>brew install openjdk@27</code>
Linux (SDKMAN)	<code>sdk install java 27-open</code>
Windows (winget)	<code>winget install Microsoft.OpenJDK.27</code>
Hepsi	Adoptium veya Oracle üzerinden indirin

Ona yönlendirme

```
# macOS
export JAVA_HOME=$(/usr/libexec/java_home -v 27)

# Linux
export JAVA_HOME=/usr/lib/jvm/jdk-27
export PATH="$JAVA_HOME/bin:$PATH"

# Windows PowerShell
$env:JAVA_HOME = "C:\Program Files\Microsoft\jdk-27"
$env:PATH = "$env:JAVA_HOME\bin;$env:PATH"
```

İlk çalıştırmanız

```
java modules/01-getting-started/examples/HelloWorld.java
```

Derleme adımı yok, build dosyası yok. Modül 01 gerçekte ne olduğunu anlatıyor.

Müfredat

Sırayla ilerleyin. Her modülün metni, öncekileri okuduğunuzu varsayar.

#	Modül	Konusu
01	getting-started	JVM, JRE ve JDK; dosyayı doğrudan çalıştırma; derleyici hatası okuma
02	variables-and-types	İlkel tipler ve referanslar, otomatik kutulama tuzakları, <code>var</code> , metin blokları
03	control-flow	Doğruluk kavramının olmayışı, switch ifadeleri, kapsayıcılık, etiketli break
04	methods	Nesneler için de geçerli olan değer geçişi, aşırı yükleme çözümlemesi, varargs
05	classes-and-objects	Klasik <code>main</code> , kapsülleme, statik fabrikalar, esnek kurucular
06	inheritance-and-polymorphism	Dinamik gönderim, <code>abstract</code> ile <code>final</code> , <code>equals</code> / <code>hashCode</code> sözleşmesi
07	interfaces	Yetenekler, varsayılan metotlar, elmas çakışması, kırılğan temel sınıflar
08	records-and-enums	Record'lar, kompakt kurucular, yüzeysel değişmezlik, davranış taşıyan enum'lar
09	sealed-and-pattern-matching	Mühürlü tipler, record desenleri, koşullar, kapsayıcılığın neden kazanç olduğu
10	exceptions	Denetlenen ve denetlenmeyen istisnalar, try-with-resources, hataların kaybolduğu üç yol
11	generics	Tip silme ve yasakladıkları, yığın kirlenmesi, PECS, değişmezlik
12	collections	Uygulama seçimi, sıralı koleksiyonlar, anahtarların neden değişmemesi gerektiği
13	lambdas	Metot referansı biçimleri, yakalama ve etkin finallik, standart arayüzler
14	streams	Tembellik, collector'lar, gatherer'lar ve paralel veri yarışı
15	optional	Ne işe yaradığı, <code>orElse</code> ile <code>orElseGet</code> farkı, beş yanlış kullanım
16	files-and-io	<code>Path</code> ile <code>Files</code> , yol geçişi saldırısı, akış halinde okuma, <code>java.io.File</code> neden değil
17	standard-library-tour	String'ler, <code>java.time</code> , yaz saati, <code>Math</code> tuzakları, tekrarlanabilir rastgelelik
18	concurrency-basics	Yarış ve görünürlük ayrı tehlikeler, <code>synchronized</code> , atomikler, executor'lar
19	virtual-threads	Taşıyıcılar ve inme, kıyaslama, bugünkü sabitleme, kapsam değerleri
20	packages-and-modules	Dizin kuralı, sınıf yolu, jar'lar ve <code>--add-opens</code> neden var
21	testing-your-code	<code>assert</code> neden test aracı değil; 39 satırda bir test çatısı
22	whats-new	JDK 26 ve 27 gerçekte ne değiştirdi, LTS, ve JEP nasıl okunur

Bunu nasıl kullanmalı

Modül README'sini okuyun, örnekleri çalıştırın, sonra ödevi çözümü açmadan **önce** yapın.

```
modules/<nn>--<ad>/
├─ README.md      ders, özet çıkarımlarla biter
├─ examples/      çalıştırılabilir dosyalar, bolca yorumlu
├─ homework/      ödev ve kabul kriterleri

solutions/<nn>--<ad>/
├─ referans çözümler, bilerek ders ağacının dışında tutuldu
```

Çözümler kendinizi karşılaştırmanız içindir, önce okumanız için değil. Her biri neden o kararları verdiğini açıklayan yorumlar taşır, dolayısıyla kendi cevabınızı ürettikten sonra okumaya değer, öncesinde ise neredeyse değersizdir.

Her ödevin açık kabul kriterleri var. Sizin sürümünüz bunları karşılıyorsa, referanstan farklı olsa bile doğrudur.

Örnekleri çalıştırma

```
# sıradan bir örnek
java modules/02-variables-and-types/examples/TheAutoboxingTrap.java

# assert gerektiren biri
java -ea modules/21-testing-your-code/examples/UsingAssertions.java

# preview özellik gerektiren biri
java --enable-preview --source 27 modules/22-whats-
new/examples/PreviewStructuredConcurrency.java
```

Bazı örnekler **bilerek** başarısız olur, çünkü kendi ürettiğiniz bir derleyici hatası, size alıntılanandan daha çok şey öğretir. Her biri ilk on satırında ne beklediğini bildirir:

İşaretleyici	Anlamı
(yok)	Derlenir ve çalışır, sıfırla çıkar
// EXPECT: compile-error	Derlenmesi başarısız olmalı ; bir derleme hatası öğretiyor
// EXPECT: runtime-error	Derlenmeli, sonra çalışma zamanında çökmeli
// EXPECT: compile-only	Derlenmeli; tek başına çalıştırılmaz
// EXPECT: assertions	-ea ile çalıştırılır, assert ifadeleri etkin olur
// EXPECT: preview	--enable-preview --source 27 ile çalıştırılır

Her şeyi doğrulama

```
./scripts/verify-examples.sh # tüm depo
./scripts/verify-examples.sh modules/09-sealed-and-pattern-matching # tek modül
```

Script her örneği ve her referans çözümü çalıştırır, her birinin işaretleyicisinde bildirdiği gibi davrandığını denetler, ve JDK 27 dışında bir sürümde hiç çalışmayı reddeder. Çünkü araç zinciri yüzünden başarısız olan bir örnek size hiçbir şey öğretmez.

Bu deponun güncel durumu:

Modül	22
Örnek dosya	105
Referans çözüm	25
Doğrulan dosya	130
Tam doğrulama süresi	yaklaşık 70 saniye

PDF

Her ders ve ödev PDF olarak da mevcut, ayrıca tüm müfredat tek bir kitap halinde.

```
./scripts/build-pdfs.sh --tr # yalnızca Türkçe
./scripts/build-pdfs.sh # her iki dil
```

Türkçe PDF'ler `pdf-tr/` klasörüne yazılır.

Proje dokümanları

- [SPEC.md](#) — bu projenin ne inşa etmeye çalıştığı ve başarı kriterleri
- [tasks/plan.md](#) — uygulama planı
- [tasks/todo.md](#) — görev dökümü, issue'lara bağlı
- [CEVIRI-NOTLARI.md](#) — çeviri kararları

Bunun nasıl inşa edildiğine dair bir not

Bu derslerdeki her olgusal iddia hatırlanarak değil üretilerek yazıldı. Derleyici hataları bozuk kod derlenerek üretildi. Kıyaslamalar ölçüldü. JEP numaraları, yayın tarihleri ve LTS bilgileri openjdk.org

üzerinden okundu.

Bu disiplin yazım sırasında gerçek hatalar yakaladı: kendi iddiasını çürüten bir kompozisyon örneği, seçilen sayıların hatayı tamamen gizlediği bir kova indeksi gösterimi, ve varsayılandan iki tur fazlası çıkan bir "yenilikler" geçmişi. Her düzeltme git geçmişinde kayıtlı.

Burada yanlış bir şey bulursanız, JEP dizini bu dokümandan uzun yaşar. Kaynağa bakın.